TP GIT — Maîtriser les commandes Git en local avec un projet React.js

Objectif : Comprendre le fonctionnement de Git, manipuler les fichiers et les commits, gérer les branches et maîtriser les commandes d'annulation et de récupération.

Tu dois développer une application React.js appelée task-manager, qui permet de gérer une liste de tâches.

L'application contient avec Les fonctionnalités suivantes:

- Afficher la liste des tâches.
- Ajouter une tâche.
- Modifier une tâche.
- Supprimer une tâche.
- Marquer une tâche comme terminée.
- Rechercher une tâche.
- Ajouter un filtre par statut.

Partie 1 — Installation et configuration de Git

Exercice 1 — Vérifier et configurer Git

1. Donner la commande permettant d'afficher la version de Git installée.
2. Donner la commande permettant de configurer globalement le nom de l'utilisateur.
3. Donner la commande permettant de configurer globalement son adresse e-mail.
4. Donner la commande permettant d'afficher la configuration Git.
5. Donner la commande permettant de définir main comme branche initiale par défaut des nouveaux dépôts.
6. Donner la commande permettant d'afficher uniquement la configuration du dépôt courant.
7. Donner la commande permettant de configurer un nom d'auteur uniquement pour le dépôt task-manager, sans modifier la configuration globale.

Exercice 2 — Créer le projet React

1. Donner la commande permettant de créer un projet React.js avec Vite nommé task-manager.
2. Donner la commande permettant d'entrer dans le dossier du projet.
3. Donner la commande permettant d'installer les dépendances.
4. Donner la commande permettant de démarrer le serveur de développement.
5. Donner la commande permettant d'initialiser un dépôt Git dans le dossier du projet.
6. Donner la commande permettant de vérifier si Git reconnaît le dépôt.
7. Donner la commande permettant d'afficher les fichiers et les répertoires non suivis par Git.
8. Expliquer la différence entre git init et git clone, en précisant dans quelle situation chaque commande est utilisée.

Partie 2 — Comprendre les trois zones de Git

On considère les trois zones suivantes :

- **Working Directory :** les fichiers présents dans le répertoire de travail.
- **Staging Area :** les modifications préparées pour le prochain commit.
- **HEAD :** le commit actuellement désigné par la référence HEAD, généralement celui de la branche active.

Exercice 3 — Premier commit du projet

Le projet React est créé et Git est initialisé. Aucun commit n'existe encore.

1. Donner la commande permettant d'afficher l'état du dépôt.
2. Donner la commande permettant d'ajouter tous les fichiers admissibles du projet à l'index.
3. Donner la commande permettant de vérifier les modifications préparées pour le premier commit.
4. Donner la commande permettant de créer le premier commit avec le message Initialisation du projet React.

5. Donner la commande permettant d'afficher uniquement le dernier commit.
6. Donner la commande permettant d'afficher l'historique complet des commits.
7. Donner la commande permettant d'afficher l'historique sous forme compacte, avec un commit par ligne.
8. Donner la commande permettant d'afficher les commits avec leur identifiant, leur auteur et leur date.

Exercice 4 — Distinguer les modifications non indexées et indexées

Le fichier src/App.jsx contient une liste de tâches. Tu effectues les opérations suivantes :

- Tu ajoutes un titre à la page.
- Tu exécutes git add src/App.jsx.
- Tu modifies ensuite le titre une deuxième fois, sans exécuter de nouveau git add.

1. Donner la commande permettant d'afficher les modifications présentes dans le répertoire de travail, mais pas encore indexées.
2. Donner la commande permettant d'afficher les modifications déjà indexées.
3. Donner la commande permettant d'afficher l'ensemble des modifications du fichier par rapport au dernier commit.
4. Donner la commande permettant de connaître les fichiers modifiés, indexés et non suivis.
5. Expliquer pourquoi la deuxième modification du titre n'est pas automatiquement intégrée à l'index.
6. Donner la commande permettant d'ajouter la deuxième modification à l'index.
7. Donner la commande permettant de comparer à nouveau l'index avec le dernier commit.
8. Donner la commande permettant de vérifier les changements qui seront inclus dans le prochain commit.

Exercice 5 — Comprendre les commandes git diff

1. Donner la commande permettant d'afficher les modifications non indexées de src/App.jsx.
2. Donner la commande permettant d'afficher les modifications indexées de src/App.jsx.

3. Donner la commande permettant de comparer le fichier de travail avec le dernier commit.
4. Donner la commande permettant de comparer deux commits identifiés par leurs SHA.
5. Donner la commande permettant de comparer les modifications entre les branches main et feature/search.
6. Expliquer la différence entre git diff et git diff --staged.
7. Expliquer pourquoi un fichier peut être modifié dans le répertoire de travail sans apparaître dans git diff --staged.

Partie 3 — Comprendre les options et les flags de Git

Exercice 6 — Les options courantes

Pour chaque situation, donner la commande correspondante.

1. Créer un commit avec un message directement dans la commande.
2. Afficher l'historique avec un identifiant abrégé par commit.
3. Afficher l'historique sous forme de graphe, avec les branches et les références.
4. Afficher les détails du dernier commit.
5. Créer une nouvelle branche et basculer dessus immédiatement.
6. Supprimer une branche locale déjà fusionnée.
7. Préparer uniquement le fichier src/App.jsx.
8. Préparer toutes les modifications admissibles du répertoire courant.
9. Afficher les modifications indexées avec l'option --staged.
10. Afficher les modifications non indexées en limitant le résultat au fichier src/App.css.

Exercice 7 — Différence entre -m, --oneline, --all et --staged

1. Donner la commande utilisant -m pour créer un commit nommé Ajout du composant TaskList.
2. Donner la commande utilisant --oneline pour consulter l'historique compact.

3. Donner la commande utilisant --all pour afficher les commits accessibles depuis toutes les références locales connues.
4. Donner la commande utilisant --staged pour afficher les changements préparés.
5. Expliquer pourquoi -m n'a pas le même rôle que --staged.
6. Expliquer pourquoi git log --oneline --all peut afficher des commits qui ne figurent pas sur la branche actuellement active.
7. Donner un exemple de commande utilisant plusieurs options Git dans une même instruction.

Partie 4 — Maîtriser git rm, git restore et les suppressions

Exercice 8 — Supprimer un fichier avec git rm

Le composant src/components/TaskItem.jsx n'est plus nécessaire.

1. Donner la commande permettant de supprimer ce fichier du répertoire de travail et de préparer sa suppression dans l'index.
2. Donner la commande permettant de vérifier que sa suppression est préparée.
3. Donner la commande permettant d'enregistrer cette suppression dans un commit.
4. Expliquer la différence entre supprimer manuellement un fichier, puis utiliser git add, et exécuter directement git rm.
5. Expliquer ce que fait git rm sur un fichier suivi par Git.
6. Le fichier est déjà supprimé manuellement. Donner la commande permettant d'enregistrer cette suppression dans l'index.
7. Donner la commande permettant de supprimer plusieurs fichiers suivis en une seule opération.

Exercice 9 — Différence entre git rm et git rm --cached

Le fichier .env contient des paramètres locaux nécessaires au développement. Il a été ajouté par erreur au dépôt Git.

1. Donner la commande permettant de retirer .env du suivi Git tout en conservant sa copie sur le disque.

2. Donner la commande permettant de retirer .env du suivi et de supprimer également sa copie locale.
3. Expliquer la différence entre ces deux commandes.
4. Le fichier .env doit désormais être ignoré. Indiquer le fichier de configuration à modifier et donner une règle permettant de l'ignorer.
5. Donner la commande permettant de vérifier si Git ignore effectivement .env.
6. Expliquer pourquoi ajouter .env à .gitignore ne suffit pas toujours à le retirer du suivi.
7. Donner la commande permettant d'enregistrer dans un commit le retrait de .env du suivi et la modification de .gitignore.
8. Si une clé secrète a déjà été publiée dans un dépôt distant, expliquer quelle action de sécurité doit être prise en plus du retrait du fichier.

Exercice 10 — Restaurer un fichier avec git restore

Tu as modifié src/App.css, mais tu souhaites abandonner les modifications non indexées et retrouver le contenu du dernier commit.

1. Donner la commande permettant de restaurer le fichier depuis l'état de l'index.
2. Donner la commande permettant de restaurer le fichier depuis le dernier commit, sans avoir préalablement modifié l'index.
3. Expliquer la différence entre git restore src/App.css et git restore --source=HEAD --worktree src/App.css.
4. Tu as modifié src/App.jsx, puis exécuté git add src/App.jsx. Donner la commande permettant de retirer ce fichier de l'index tout en conservant les modifications dans le répertoire de travail.
5. Donner la commande permettant de restaurer le contenu de src/App.jsx depuis le dernier commit à la fois dans l'index et dans le répertoire de travail.
6. Expliquer pourquoi git restore --staged src/App.jsx ne supprime normalement pas les modifications présentes uniquement dans le répertoire de travail.
7. Expliquer la différence entre restaurer un fichier et créer un commit qui annule un changement.

Partie 5 — Maîtriser git reset --soft, --mixed et --hard

Exercice 11 — Comprendre git reset --soft

Tu viens de créer un commit nommé Ajout du filtre des tâches. Après vérification, tu constates que le message est incorrect et que tu souhaites modifier le commit.

1. Donner la commande permettant de revenir d'un commit en arrière tout en conservant les modifications dans l'index.
2. Donner la commande permettant de vérifier que les changements sont toujours indexés.
3. Donner la commande permettant de recréer le commit avec le message Ajout du filtre par statut.
4. Expliquer ce que --soft modifie dans l'historique.
5. Expliquer ce que --soft conserve dans l'index et dans le répertoire de travail.
6. Donner un exemple de situation dans laquelle git reset --soft est plus adapté que git reset --hard.
7. Expliquer ce qui peut arriver si tu exécutes git reset --soft HEAD~1 alors que tu as plusieurs modifications indexées.

Exercice 12 — Comprendre git reset --mixed

Tu as créé un commit contenant une modification de src/App.jsx. Tu souhaites retirer ce commit de la tête de branche, conserver les modifications dans les fichiers et les retirer de l'index.

1. Donner la commande permettant de revenir au commit précédent en utilisant explicitement --mixed.
2. Donner la commande permettant de vérifier que les modifications sont présentes mais ne sont plus indexées.
3. Donner la commande permettant de réindexer uniquement src/App.jsx.
4. Expliquer la différence entre git reset --soft HEAD~1 et git reset --mixed HEAD~1.
5. Indiquer l'effet de --mixed sur HEAD, l'index et le répertoire de travail.
6. Donner une situation concrète dans laquelle git reset --mixed est utile.
7. Expliquer ce qui se passe si tu exécutes git reset --mixed HEAD~1 après avoir préparé plusieurs fichiers pour le prochain commit.

Exercice 13 — Comprendre git reset --hard

Tu as créé un commit incorrect contenant des modifications de src/App.jsx. Tu souhaites revenir au commit précédent et abandonner les changements concernés.

1. Donner la commande permettant de revenir au commit précédent avec --hard.
2. Indiquer ce que cette commande fait à HEAD, à l'index et au répertoire de travail.
3. Expliquer pourquoi cette commande est plus risquée que git reset --soft et git reset --mixed.
4. Donner la commande permettant d'afficher l'état du dépôt avant d'envisager une réinitialisation destructive.
5. Expliquer pourquoi un fichier non suivi n'est pas nécessairement supprimé par git reset --hard.
6. Donner une méthode permettant de sécuriser les modifications utiles avant de réinitialiser le dépôt.
7. Expliquer pourquoi il ne faut pas utiliser systématiquement git reset --hard HEAD~1 pour corriger un commit incorrect.
8. Un commit a déjà été partagé avec l'équipe. Expliquer pourquoi réécrire l'historique partagé peut poser problème et indiquer une alternative plus adaptée.

Exercice 14 — Comparaison complète des trois modes de git reset

On considère le même état initial pour chaque scénario :

- Le dernier commit modifie src/App.jsx.
- Le fichier contient des modifications supplémentaires dans le répertoire de travail.
- Certaines modifications sont déjà indexées.
- Le commit précédent est désigné par HEAD~1.

Pour chaque scénario, donner la commande et préciser l'état final de HEAD, de l'index et du répertoire de travail.

1. Revenir au commit précédent tout en conservant les modifications indexées.
2. Revenir au commit précédent en conservant les modifications dans les fichiers, mais en réinitialisant l'index.
3. Revenir au commit précédent en abandonnant les modifications suivies concernées.

4. Expliquer pourquoi --soft, --mixed et --hard ne sont pas interchangeables.
5. Indiquer quel mode choisir pour corriger uniquement le message du dernier commit.
6. Indiquer quel mode choisir pour recommencer la préparation des fichiers avant un nouveau commit.
7. Indiquer quel mode présente le risque le plus élevé de perdre des modifications locales non sauvegardées.

Partie 6 — Comprendre l'historique avec git log, git show et git reflog

Exercice 15 — Utiliser git log

1. Donner la commande permettant d'afficher l'historique complet de la branche active.
2. Donner la commande permettant d'afficher uniquement le dernier commit.
3. Donner la commande permettant d'afficher les cinq derniers commits en format compact.
4. Donner la commande permettant d'afficher l'historique avec un graphe et les noms des branches.
5. Donner la commande permettant d'afficher l'historique de toutes les branches locales et des références connues.
6. Donner la commande permettant d'afficher les commits contenant des modifications de src/App.jsx.
7. Donner la commande permettant de rechercher dans les messages de commit ceux qui contiennent le mot task.
8. Donner la commande permettant d'afficher les commits réalisés par un auteur précis.
9. Donner la commande permettant d'afficher les commits effectués depuis une date donnée.
10. Donner la commande permettant d'afficher les commits qui sont sur feature/search mais pas sur main.
11. Expliquer la différence entre git log, git show et git status.

Exercice 16 — Examiner un commit précis

Tu veux analyser un commit qui a ajouté le composant TaskList.

1. Donner la commande permettant d'afficher les détails du dernier commit.
2. Donner la commande permettant d'afficher les détails d'un commit identifié par son SHA.
3. Donner la commande permettant d'afficher le résumé des fichiers modifiés dans un commit.
4. Donner la commande permettant d'afficher les différences introduites par un commit.
5. Donner la commande permettant d'afficher uniquement le message et les métadonnées d'un commit, sans afficher le patch complet.
6. Expliquer pourquoi le SHA d'un commit est utile pour retrouver une modification précise.

Exercice 17 — Récupérer une référence perdue avec git reflog

Tu as exécuté une commande git reset, puis tu constates qu'un commit local que tu souhaitais conserver n'apparaît plus dans l'historique habituel.

1. Donner la commande permettant d'afficher les déplacements récents de HEAD.
2. Donner la commande permettant d'afficher les dernières entrées du reflog.
3. Après avoir retrouvé le SHA du commit recherché, donner la commande permettant de créer une branche de récupération à partir de ce commit.
4. Expliquer pourquoi git log et git reflog n'affichent pas nécessairement les mêmes informations.
5. Expliquer pourquoi le reflog peut aider à retrouver un commit devenu inaccessible depuis la branche courante.
6. Expliquer pourquoi il faut agir rapidement lorsqu'on cherche à récupérer un commit perdu et pourquoi le reflog ne constitue pas une sauvegarde permanente.

Partie 7 — Gérer les fichiers ignorés et nettoyer le projet

Exercice 18 — Configurer .gitignore

Le projet React utilise Vite et génère des fichiers locaux ou temporaires.

1. Donner le nom du fichier permettant de définir les règles d'exclusion de Git.
2. Donner une règle permettant d'ignorer node_modules.

3. Donner une règle permettant d'ignorer le dossier dist.
4. Donner une règle permettant d'ignorer les fichiers .env.
5. Donner une règle permettant d'ignorer les fichiers .log.
6. Donner la commande permettant d'afficher les fichiers ignorés par Git.
7. Donner la commande permettant de vérifier quelle règle ignore un fichier particulier.
8. Expliquer pourquoi .gitignore n'annule pas automatiquement le suivi d'un fichier déjà versionné.

Exercice 19 — Comprendre git clean

Le répertoire de travail contient des fichiers non suivis : src/TestTemp.jsx et notes-temp.txt.

1. Donner la commande permettant d'afficher les fichiers non suivis sans les supprimer.
2. Donner la commande permettant de simuler le nettoyage des fichiers non suivis.
3. Donner la commande permettant de supprimer les fichiers non suivis après avoir vérifié le résultat de la simulation.
4. Expliquer la différence entre git clean et git rm.
5. Expliquer pourquoi il faut être prudent avec les options -f et -d.
6. Expliquer la différence entre supprimer des fichiers non suivis et supprimer des fichiers suivis dont les modifications ne sont pas validées.

Partie 8 — Travailler avec les branches locales

Exercice 20 — Créer et consulter les branches

La branche principale contient la version stable de task-manager.

1. Donner la commande permettant d'afficher les branches locales.
2. Donner la commande permettant de créer une branche nommée feature/add-task.
3. Donner la commande permettant de basculer sur cette branche.
4. Donner la commande permettant de créer la branche et de basculer dessus en une seule opération.
5. Donner la commande permettant de connaître la branche actuellement active.

6. Donner la commande permettant de renommer la branche locale courante.
7. Donner la commande permettant de supprimer une branche locale déjà fusionnée.
8. Donner la commande permettant de forcer la suppression d'une branche locale non fusionnée, et expliquer pourquoi cette opération nécessite de la prudence.
9. Expliquer la différence entre git branch et git switch.

Exercice 21 — Développer une fonctionnalité sur une branche

Tu dois ajouter une fonctionnalité de recherche des tâches sans modifier directement main.

1. Donner la commande permettant de créer feature/search-task à partir de la branche courante et de basculer dessus.
2. Donner la commande permettant d'afficher l'état du dépôt avant de modifier le code.
3. Donner les commandes permettant de préparer uniquement src/App.jsx et src/App.css.
4. Donner la commande permettant de créer un commit nommé Ajout de la recherche des tâches.
5. Donner la commande permettant de consulter l'historique de la branche.
6. Donner la commande permettant de revenir sur main.
7. Donner la commande permettant d'intégrer feature/search-task dans main.
8. Donner la commande permettant de supprimer la branche de fonctionnalité après une fusion réussie.
9. Expliquer pourquoi une branche dédiée facilite le développement et la vérification d'une fonctionnalité.

Exercice 22 — Fusionner deux branches locales

La branche feature/add-task ajoute la création de tâches. La branche feature/search-task ajoute la recherche.

1. Donner la commande permettant de revenir sur main.
2. Donner la commande permettant de fusionner feature/add-task dans main.
3. Donner la commande permettant de fusionner feature/search-task dans main.
4. Donner la commande permettant de créer un commit de fusion même si un fast-forward est possible.

5. Donner la commande permettant d'afficher le graphe de l'historique après les fusions.
6. Expliquer la différence entre une fusion fast-forward et une fusion avec un commit de merge.
7. Expliquer pourquoi il faut vérifier l'état du dépôt avant de changer de branche ou de fusionner.

Partie 9 — Résoudre les conflits et réorganiser l'historique

Exercice 23 — Simuler un conflit

Deux branches modifient la même ligne de src/App.jsx.

- feature/add-task remplace le titre par « Gestionnaire de tâches ».
- feature/search-task remplace cette même ligne par « Rechercher une tâche ».

1. Donner les commandes permettant de créer les deux branches à partir d'un même commit.
2. Donner les commandes permettant de modifier le fichier et de créer un commit sur chaque branche.
3. Donner la commande permettant de revenir sur main.
4. Donner la commande permettant de fusionner une première branche.
5. Donner la commande permettant de fusionner la seconde branche et de provoquer le conflit.
6. Donner la commande permettant d'identifier les fichiers en conflit.
7. Après avoir corrigé manuellement src/App.jsx, donner la commande permettant de préparer le fichier résolu.
8. Donner la commande permettant de terminer la fusion si Git attend encore un commit de merge.
9. Donner la commande permettant de vérifier l'historique après la résolution.
10. Expliquer pourquoi Git ne peut pas toujours choisir automatiquement entre deux modifications concurrentes.

Exercice 24 — Utiliser git rebase

La branche feature/search-task contient des commits locaux. Entre-temps, main a reçu de nouveaux commits.

1. Donner la commande permettant de basculer sur feature/search-task.
2. Donner la commande permettant de replacer les commits de cette branche au-dessus de la version actuelle de main.
3. Donner la commande permettant de consulter l'état du dépôt lorsqu'un conflit survient pendant le rebase.
4. Après avoir résolu un conflit, donner la commande permettant de marquer le fichier comme résolu.
5. Donner la commande permettant de poursuivre le rebase.
6. Donner la commande permettant d'abandonner le rebase en cours.
7. Expliquer la différence entre git merge main et git rebase main exécutés depuis feature/search-task.
8. Expliquer pourquoi les identifiants des commits peuvent changer après un rebase.
9. Expliquer pourquoi il faut éviter de réécrire sans coordination l'historique d'une branche partagée.

Partie 10 — Mettre de côté son travail avec git stash

Exercice 25 — Sauvegarder temporairement des modifications

Tu travailles sur feature/add-task. Tu as modifié src/App.jsx, mais tu dois basculer immédiatement sur une autre branche sans créer de commit.

1. Donner la commande permettant de mettre de côté les modifications suivies avec le message Travail temporaire sur add-task.
2. Donner la commande permettant d'afficher la liste des stash enregistrés.
3. Donner la commande permettant de consulter le contenu d'un stash.
4. Donner la commande permettant de réappliquer le stash le plus récent tout en le conservant dans la liste.
5. Donner la commande permettant de réappliquer le stash le plus récent et de le retirer généralement de la liste lorsque l'application réussit.

6. Donner la commande permettant de supprimer explicitement une entrée de stash après vérification.
7. Expliquer la différence entre git stash apply et git stash pop.
8. Expliquer pourquoi un stash ne remplace pas un commit permanent.

Partie 11 — Tags et versions locales

Exercice 26 — Marquer une version stable

La version 1.0.0 de task-manager est prête.

1. Donner la commande permettant de créer un tag annoté v1.0.0.
2. Donner la commande permettant d'afficher la liste des tags.
3. Donner la commande permettant d'afficher les détails du tag v1.0.0.
4. Donner la commande permettant de créer un tag sur un commit précis identifié par son SHA.
5. Donner la commande permettant de supprimer un tag local nommé v1.0.0.
6. Expliquer la différence entre un tag et une branche.
7. Expliquer pourquoi un tag annoté est utile pour identifier une version stable du projet.

Partie 12 — Annuler des changements sans perdre son travail

Exercice 27 — Choisir la bonne commande d'annulation

Pour chaque situation, donner la commande la plus adaptée.

1. Tu as modifié src/App.jsx sans exécuter git add et tu veux abandonner ces modifications.
2. Tu as exécuté git add src/App.jsx, mais tu ne souhaites plus inclure ce fichier dans le prochain commit. Tu veux conserver ses modifications.
3. Tu viens de créer un commit local et tu souhaites revenir en arrière tout en conservant les changements indexés.
4. Tu veux retirer le dernier commit local et conserver les changements dans les fichiers, mais sans les laisser indexés.

5. Tu veux retirer le dernier commit local et abandonner les modifications suivies correspondantes.
6. Tu as supprimé accidentellement un fichier suivi et tu veux retrouver sa version du dernier commit.
7. Tu veux annuler les effets d'un commit déjà partagé avec ton équipe sans réécrire l'historique partagé.
8. Tu veux retrouver un commit qui n'apparaît plus dans l'historique normal après un reset.
9. Tu veux appliquer à la branche courante les changements d'un commit précis provenant d'une autre branche.
10. Tu souhaites enregistrer tes modifications temporaires sans créer de commit.

Exercice 28 — Comparer les méthodes d'annulation

1. Expliquer la différence entre git restore et git reset.
2. Expliquer la différence entre git reset --soft et git reset --mixed.
3. Expliquer la différence entre git reset --mixed et git reset --hard.
4. Expliquer la différence entre git reset --hard HEAD~1 et git revert HEAD.
5. Donner une situation dans laquelle git restore --staged est préférable à git reset --hard.
6. Donner une situation dans laquelle git revert est préférable à git reset.
7. Expliquer pourquoi un commit supprimé de la tête d'une branche n'est pas nécessairement immédiatement irrécupérable.
8. Proposer une méthode prudente pour annuler un commit local tout en conservant une possibilité de récupération.

Partie 13 — Commandes complémentaires du travail local

Exercice 29 — Identifier les fichiers et les modifications

1. Donner la commande permettant d'afficher les fichiers modifiés, indexés et non suivis.
2. Donner la commande permettant d'afficher l'état du dépôt dans un format court.
3. Donner la commande permettant de vérifier les différences non indexées d'un seul fichier.

4. Donner la commande permettant de vérifier les différences indexées d'un seul fichier.
5. Donner la commande permettant d'afficher les fichiers suivis par Git.
6. Donner la commande permettant d'afficher les fichiers ignorés.
7. Donner la commande permettant d'afficher les modifications introduites par le dernier commit.
8. Donner la commande permettant de comparer deux versions du même fichier à partir de deux commits distincts.

Exercice 30 — Comprendre git commit et ses options

1. Donner la commande permettant de créer un commit avec un message explicite.
2. Donner la commande permettant de modifier le message du dernier commit local.
3. Donner la commande permettant d'intégrer au dernier commit des modifications déjà indexées, sans créer un nouveau commit.
4. Expliquer la différence entre créer un nouveau commit et modifier le dernier commit avec --amend.
5. Expliquer pourquoi --amend doit être utilisé avec prudence lorsque le commit a déjà été partagé.
6. Donner la commande permettant de créer un commit qui ne contient que les modifications préparées d'un fichier précis, lorsque cette utilisation est appropriée.
7. Expliquer pourquoi il faut examiner git diff --staged avant de créer un commit important.

Exercice 31 — Annuler un commit avec git revert

Le dernier commit a introduit un bug qui empêche l'ajout d'une tâche. Le commit a déjà été partagé avec l'équipe.

1. Donner la commande permettant de créer un nouveau commit qui inverse les changements du dernier commit.
2. Donner la commande permettant d'annuler un commit précis à partir de son SHA.
3. Donner la commande permettant de consulter le nouvel historique après l'annulation.
4. Expliquer pourquoi git revert conserve le commit initial dans l'historique.
5. Expliquer pourquoi cette méthode est généralement préférable à un reset qui réécrit une branche partagée.

Exercice 32 — Appliquer un commit précis avec git cherry-pick

La branche feature/search-task contient un correctif utile. Tu souhaites appliquer ce correctif sur main sans fusionner toute la branche.

1. Donner la commande permettant de retrouver le SHA du commit à appliquer.
2. Donner la commande permettant d'appliquer ce commit sur la branche courante.
3. Donner la commande permettant de consulter l'historique après l'opération.
4. Expliquer ce qui se passe si le commit provoque un conflit.
5. Expliquer la différence entre git cherry-pick et git merge.

Partie 14 — Défi final : réparer et livrer Task Manager

Exercice 33 — Scénario complet

Tu développes task-manager en local. Le dépôt contient déjà plusieurs commits et les branches main, feature/add-task et feature/search-task.

La situation est la suivante :

- Tu as modifié src/App.jsx, mais tu n'as pas encore indexé toutes les modifications.
- Tu as préparé src/App.css pour le prochain commit.
- Tu as créé un commit dont le message est incorrect.
- Tu souhaites conserver le code développé, mais revoir la préparation des fichiers.
- Tu dois ensuite intégrer la recherche dans main.
- Enfin, tu dois marquer la version stable 1.0.0.

Répondre aux questions suivantes en donnant les commandes adaptées.

1. Afficher l'état complet du dépôt.
2. Afficher les modifications non indexées de src/App.jsx.
3. Afficher les modifications indexées de src/App.css.
4. Retirer src/App.css de l'index tout en conservant son contenu modifié.
5. Retirer le dernier commit de la tête de branche tout en conservant les changements indexés.

6. Recréer le commit avec un message correct.
7. Afficher les cinq derniers commits en format compact.
8. Afficher le graphe des commits et les branches connues.
9. Créer une branche feature/fix-task-search et basculer dessus.
10. Préparer uniquement les fichiers nécessaires au correctif.
11. Créer un commit nommé Correction de la recherche des tâches.
12. Revenir sur main.
13. Fusionner la branche de correctif dans main.
14. Vérifier l'historique après la fusion.
15. Créer un tag annoté v1.0.0.
16. Afficher les détails de ce tag.
17. Si tu constates que le dernier commit contient une erreur après avoir déjà publié la version avec ton équipe, choisir une méthode d'annulation adaptée sans réécrire l'historique partagé.
18. Si tu perds ensuite la trace d'un commit local après une réinitialisation, indiquer la commande permettant de rechercher les déplacements récents de HEAD.

Partie 15 — Synthèse finale : quelle commande utiliser ?

Pour chaque besoin, écrire la commande Git correspondante.

1. Vérifier l'état du dépôt.
2. Initialiser un dépôt.
3. Cloner un dépôt existant.
4. Préparer un fichier.
5. Préparer tous les fichiers admissibles.
6. Afficher les modifications non indexées.
7. Afficher les modifications indexées.
8. Créer un commit.

9. Consulter l'historique compact.
10. Afficher les détails d'un commit.
11. Supprimer un fichier suivi et préparer sa suppression.
12. Retirer un fichier du suivi tout en conservant sa copie locale.
13. Abandonner les modifications non indexées d'un fichier.
14. Retirer un fichier de l'index sans perdre ses modifications.
15. Revenir d'un commit en conservant les changements indexés.
16. Revenir d'un commit en conservant les changements dans les fichiers, mais sans les indexer.
17. Revenir d'un commit en abandonnant les modifications suivies concernées.
18. Rechercher les déplacements récents de HEAD.
19. Créer une branche.
20. Changer de branche.
21. Fusionner une branche.
22. Mettre temporairement des modifications de côté.
23. Réappliquer un stash sans le supprimer.
24. Créer un tag annoté.
25. Annuler les effets d'un commit partagé.
26. Appliquer un commit précis provenant d'une autre branche.
27. Afficher les fichiers non suivis sans les supprimer.
28. Simuler le nettoyage des fichiers non suivis.
29. Afficher le graphe de l'historique.
30. Comparer deux commits.